

Skript zu Kapitel 6

Transmission Control Protocol

Vollständige Ausarbeitung des Foliensatzes mit ergänzenden sachlichen Erklärungen

Lehrveranstaltung	VO Computernetzwerke
Kapitel	6 - Transmission Control Protocol
Semester	SS 2026
Foliensatz	Armin Veichtlbauer

Inhaltlicher Schwerpunkt: TCP-Grundlagen, ARQ-Verfahren, TCP Sliding Window, Flow Control und Congestion Control

Hinweis: Die Ausarbeitung enthält sämtliche sichtbaren Inhalte der 21 PDF-Seiten einschließlich der kleinen Zusatznotizen auf den Folien. Technisch missverständliche Aussagen werden gekennzeichnet und sachlich eingeordnet.

Inhaltsübersicht

1. Überblick und Lernziele

2. TCP-Grundlagen

- 2.1 Eigenschaften und Einsatzgebiet
- 2.2 Zuverlässigkeit und Grenzen
- 2.3 Aufbau des TCP-Headers
- 2.4 Steuerungsflags
- 2.5 TCP-3-Way-Handshake

3. Automated Repeat Request (ARQ)

- 3.1 Grundidee und Fehlererkennung
- 3.2 Stop-and-Wait
- 3.3 Schiebefensterverfahren
- 3.4 Kumulative und selektive Bestätigungen
- 3.5 Fehlerbehandlung, Go-Back-N und Selective Repeat

4. TCP Sliding Window

- 4.1 Sequenzraum und Zustandsvariablen
- 4.2 Ablauf der Datenübertragung
- 4.3 Festlegung der Fenstergröße und Channel Utilization
- 4.4 Flow Control und Congestion Control
- 4.5 Slow Start und Verlauf des Congestion Windows

5. Kompakte Zusammenfassung

6. Begriffs- und Formelübersicht

Leselogik: Die Begriffe werden zuerst entsprechend den Folien wiedergegeben. Danach folgen einfache sachliche Ergänzungen. Es werden keine Analogien verwendet.

1. Überblick und Lernziele

Kapitel 6 behandelt das Transmission Control Protocol (TCP) in drei Themenblöcken: TCP-Grundlagen, ARQ-Verfahren und Überlaststeuerungsmechanismen in TCP.

1.1 Fragen zu den TCP-Grundlagen

- Wie funktioniert TCP?
- Welche Headerfelder besitzt TCP und wozu dienen sie?
- Wie funktioniert der TCP-3-Way-Handshake?

1.2 Fragen zu ARQ

- Wie funktioniert ARQ?
- Welche ARQ-Verfahren können unterschieden werden?
- Was sind Schiebefensterverfahren und wie funktionieren sie?

1.3 Fragen zur Überlaststeuerung in TCP

- Welches ARQ-Verfahren verwendet TCP?
- Wie kann das ARQ-Prinzip für die Überlaststeuerung verwendet werden?
- Wie werden die Fenstergrößen bestimmt und berechnet?

Ergebnis des Kapitels: TCP transportiert einen zuverlässigen Bytestrom. Die Zuverlässigkeit entsteht durch Sequenznummern, Bestätigungen, Zeitüberwachung und Wiederholungen. Die Menge gleichzeitig unbestätigter Daten wird durch Fenster begrenzt. Das tatsächlich nutzbare Sendefenster ist der kleinere Wert aus Empfängerfenster und Congestion Window.

Abgedeckte Folien: Titelfolie und Überblick (PDF-Seiten 1-2).

2. TCP-Grundlagen

2.1 Eigenschaften und Einsatzgebiet

- Das Transmission Control Protocol ist im ursprünglichen Foliensatz mit RFC 793 als Basis angegeben; ergänzend werden RFC 1122 und RFC 1323 genannt.
- TCP stellt eine End-to-End-Verbindung über ein IP-Netz her.
- TCP wird als Transportdienst für verteilte Anwendungen eingesetzt, die einen zuverlässigen und verbindungsorientierten Transport benötigen. Genannte Beispiele sind File Transfer, File Sharing, E-Mail und Webbrowsing.
- Das Protokoll implementiert Flow-Control- und Congestion-Control-Algorithmen.
- TCP transportiert einen Bytestrom über eine logische Verbindung, die in den Folien als Virtual Circuit bezeichnet wird. Der Sender unterteilt den Bytestrom in Segmente; der Empfänger setzt die Daten wieder zum Bytestrom zusammen.

Sachliche Erklärung: End-to-End bedeutet, dass die TCP-Zustände an den beiden Kommunikationsendpunkten geführt werden. IP-Router dazwischen leiten IP-Pakete weiter, verwalten aber nicht die TCP-Verbindung. Ein Virtual Circuit ist hier eine logisch zustandsbehaftete Verbindung; es wird keine physische Leitung reserviert.

Wichtig für Sequenznummern: TCP nummeriert Bytes und nicht lediglich Segmente. Ein Segment trägt als Sequence Number die Nummer seines ersten Nutzdatenbytes. Dadurch können unterschiedlich große Segmente korrekt in den ursprünglichen Bytestrom eingeordnet werden.

Abgedeckte Folie: Eigenschaften von TCP (PDF-Seite 3 / Folie 2).

VO Computernetzwerke - Kapitel 6: TCP Seite 3

2.2 Zuverlässigkeit und Grenzen

TCP stellt Zuverlässigkeit durch mehrere zusammenwirkende Mechanismen bereit:

- Empfangsbestätigung: Der Empfänger bestätigt korrekt erhaltene Daten mit ACKs.
- Erneute Übertragung: Wird ein Segment nicht rechtzeitig bestätigt, erkennt der Sender dies über einen Timeout und überträgt die Daten erneut. Dieser Vorgang heißt Retransmission und beruht auf ARQ.
- Duplikaterkennung: Eindeutige Sequenznummern ermöglichen es, mehrfach eingetroffene Daten zu erkennen.
- Reihenfolge: Aufsteigende Sequenznummern ermöglichen die Wiederherstellung der ursprünglichen Byte-Reihenfolge.
- Integrität: Eine Prüfsumme erkennt Übertragungsfehler; das Prinzip entspricht der UDP-Prüfsumme.
- Keine Angriffssicherheit: TCP selbst bietet weder Verschlüsselung noch Authentifizierung.

Sachliche Erklärung: Zuverlässigkeit und Sicherheit sind unterschiedliche Eigenschaften. TCP kann Datenverluste, Duplikate, Reihenfolgefehler und bestimmte Bitfehler behandeln. Es schützt den Inhalt jedoch nicht vor Mitlesen und weist die Identität des Kommunikationspartners nicht kryptografisch nach.

Abgedeckte Folie: Zuverlässigkeit von TCP (PDF-Seite 4 / Folie 3).

2.3 Aufbau des TCP-Headers

Der TCP-Header enthält feste Felder und optional zusätzliche Optionen. Die folgende Originalabbildung zeigt die Anordnung in 32-Bit-Zeilen.

6.1 TCP Grundlagen

TCP Header

0 - 3	4 - 7	8 - 11	12 - 15	16 - 19	20 - 23	24 - 27	28 - 31
Source Port				Destination Port			
Sequence Number (SN)							
Acknowledge Number (AN)							
Offset	Reserved	Flags		Window (WND)			
Checksum				Urgent Pointer			
Options und Padding (optional)*							

* Options können auch mehrere 32 bit Worte umfassen (aber immer nur ganze Worte)



Abbildung 1: TCP-Header aus dem Foliensatz, einschließlich Bitpositionen und Hinweis zu Options/Padding (Original aus PDF-Seite 5 / Folie 4).

Headerfeld	Bedeutung	Ergänzung
Source Port	Portnummer der sendenden Anwendung.	Wie bei UDP; zusammen mit IP-Adressen und Zielpport zur Zuordnung einer Verbindung.
Destination Port	Portnummer der empfangenden Anwendung.	Wie bei UDP.
Sequence Number (SN)	Nummer des ersten Bytes dieses TCP-Segments im ursprünglichen Bytestrom.	SYN und FIN verbrauchen ebenfalls Sequenzraum.

Headerfeld	Bedeutung	Ergänzung
Acknowledge Number (AN)	Nummer des ersten Bytes, das noch nicht bestätigt ist.	Bestätigt damit kumulativ alle vorherigen Bytes.
Offset	Länge des TCP-Headers in 32-Bit-Worten.	Ohne Optionen beträgt der Wert 5, also 20 Byte.
Reserved	Für Erweiterungen reservierte Bits.	Im normalen Betrieb auf den vorgeschriebenen Wert setzen.
Flags	Steuerungsbits URG, ACK, PSH, RST, SYN und FIN.	Regeln Verbindungsaufbau, Bestätigungen, Abbau und Sonderfälle.
Window (WND)	Größe des vom Empfänger angekündigten Empfangsfensters.	So viele Bytes dürfen im Rahmen der Flow Control unbestätigt sein.
Checksum	Prüfsumme über TCP-Header, Nutzdaten und Pseudoheader.	Wie bei UDP wird ein Pseudoheader verwendet.
Urgent Pointer	Kennzeichnet bei gesetztem URG-Flag die Grenze des dringenden Bereichs am Beginn der Nutzdaten.	Das Feld ist nur zusammen mit URG relevant.
Options und Padding	Optionale Zusatzinformationen; Padding richtet den Header auf 32-Bit-Grenzen aus.	Optionen können mehrere 32-Bit-Worte umfassen, aber nur ganze Worte.

Pseudoheader: Der Pseudoheader ist kein tatsächlich vor dem TCP-Header übertragenes TCP-Feld. Er bezieht Informationen aus der IP-Schicht in die Prüfsumme ein, damit unter anderem falsche Zuordnungen zu IP-Adressen oder Protokollnummern erkannt werden können.

Abgedeckte Folien: TCP-Header und Bedeutung der Headerfelder (PDF-Seiten 5-6 / Folien 4-5).

2.4 Steuerungsflags

Flag	Funktion
URG - Urgent	Ermöglicht der Anwendung, den als dringend markierten Nutzdatenbereich bis zum Urgent Pointer sofort zu lesen und entsprechend zu reagieren.
ACK - Acknowledge	Kennzeichnet, dass die Acknowledge Number gültig ist und bestätigt die korrekte Ankunft von Daten bis zu diesem Wert.
PSH - Push	Zeigt an, dass vorhandene Daten möglichst ohne weiteres Sammeln an die empfangende Anwendung weitergereicht werden sollen. Dies kann die Bearbeitung zeitkritischer Daten beschleunigen.
RST - Reset	Setzt eine Verbindung ohne regulären Handshake zurück, beispielsweise bei technischen Problemen oder zur Abweisung einer unerwünschten Verbindung.
SYN - Synchronize	Kennzeichnet eine Anfrage zum Verbindungsaufbau und dient der Synchronisation der initialen Sequenznummern.
FIN - Finish	Leitet den geordneten Verbindungsabbau mit dem üblichen FIN-Handshake ein.

Sachliche Ergänzung zu PSH: PSH erhöht nicht die Übertragungsgeschwindigkeit im Netz. Das Flag betrifft vor allem die zeitnahe Übergabe bereits empfangener Daten an die Anwendung.

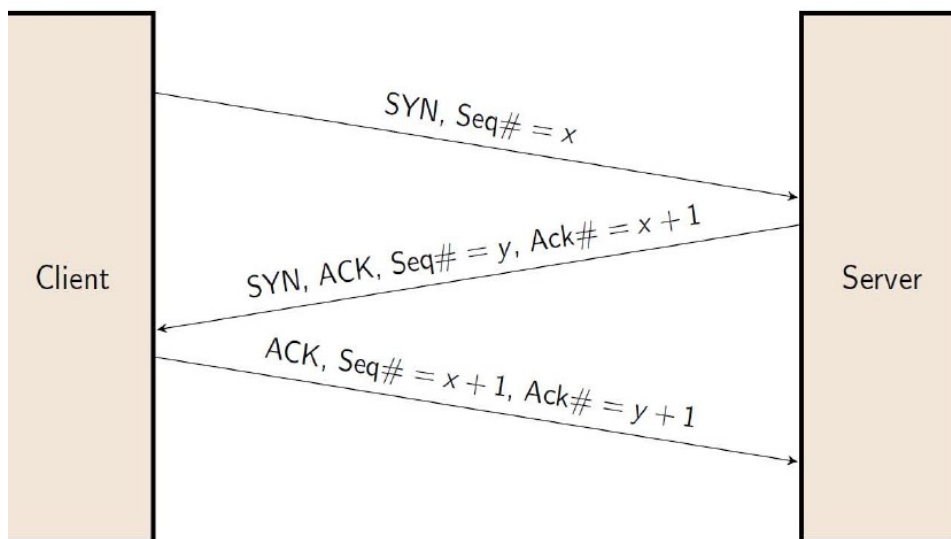
Sachliche Ergänzung zu FIN: TCP ist voll-duplex. Jede Richtung des Bytestroms wird getrennt beendet. Deshalb besteht ein vollständiger geordneter Abbau häufig aus einem FIN/ACK-Austausch für jede Richtung.

Abgedeckte Folie: Flags (PDF-Seite 7 / Folie 6).

2.5 TCP-3-Way-Handshake

Vor dem Datentransport wird eine TCP-Verbindung mit drei Nachrichten aufgebaut:

1. Client -> Server: SYN mit der initialen Sequenznummer des Clients x .
2. Server -> Client: SYN und ACK mit der initialen Sequenznummer des Servers y sowie Acknowledge Number $x + 1$.
3. Client -> Server: ACK mit Sequence Number $x + 1$ und Acknowledge Number $y + 1$. Diese dritte Nachricht kann bereits Daten enthalten.



TCP 3-Way Handshake

Abbildung 2: TCP-3-Way-Handshake mit den initialen Sequenznummern x und y (Original aus PDF-Seite 8 / Folie 7).

Warum drei Nachrichten erforderlich sind: Beide Seiten müssen ihre eigene initiale Sequenznummer mitteilen und die Sequenznummer der Gegenseite bestätigen. Nach der dritten Nachricht ist für beide Seiten erkennbar, dass die Synchronisation in beide Richtungen erfolgreich war.

Korrekturhinweis zur Folie: Im dritten Aufzählungspunkt der Folie steht „SN des Servers ($x+1$)“. Die dritte Nachricht wird jedoch vom Client gesendet; das Diagramm zeigt korrekt $\text{Seq} = x + 1$ und $\text{Ack} = y + 1$.

Abgedeckte Folie: TCP-Verbindungsaufbau (PDF-Seite 8 / Folie 7).

3. Automated Repeat Request (ARQ)

3.1 Grundidee und Fehlererkennung

ARQ sorgt durch Bestätigungen und Wiederholungen für eine zuverlässige Übertragung.

- Wenn ein Segment richtig empfangen wird, wird es bestätigt.
- Wenn ein Segment nicht richtig empfangen wird, wird es erneut gesendet.
- Verlorene Segmente müssen durch Nummerierung erkennbar sein.

Die Folien unterscheiden zwei Formen der Fehlerentdeckung:

- Implizite Fehlerentdeckung: Ein Timeout läuft ab, weil keine passende Bestätigung rechtzeitig eingetroffen ist.
- Explizite Fehlerentdeckung: Der Empfänger sendet ein NAK zurück, beispielsweise nach einem erkannten CRC-Fehler.
- Wichtige Einschränkung: Ein NAK allein funktioniert nicht zuverlässig bei Segmentverlust. Ein vollständig verlorenes Segment kann vom Empfänger nicht unmittelbar beanstandet werden, wenn er keine Information über dessen Versand erhält.

Timeout: Der Sender startet für noch nicht bestätigte Daten eine Zeitüberwachung. Läuft sie ab, wird angenommen, dass Daten oder Bestätigung verloren gegangen sind. Die betroffenen Daten werden erneut übertragen.

ACK und NAK: Ein ACK bestätigt korrekte Daten. Ein NAK meldet ausdrücklich einen erkannten Fehler. TCP arbeitet hauptsächlich mit ACKs, Sequenznummern, Timeouts und wiederholten ACKs; ein eigenes allgemeines TCP-NAK-Flag ist nicht Bestandteil des hier gezeigten Headers.

Abgedeckte Folie: ARQ-Grundidee (PDF-Seite 9 / Folie 8).

3.2 Arten von ARQ-Verfahren

Die Folien unterscheiden Stop-and-Wait und Schiebefensterverfahren.

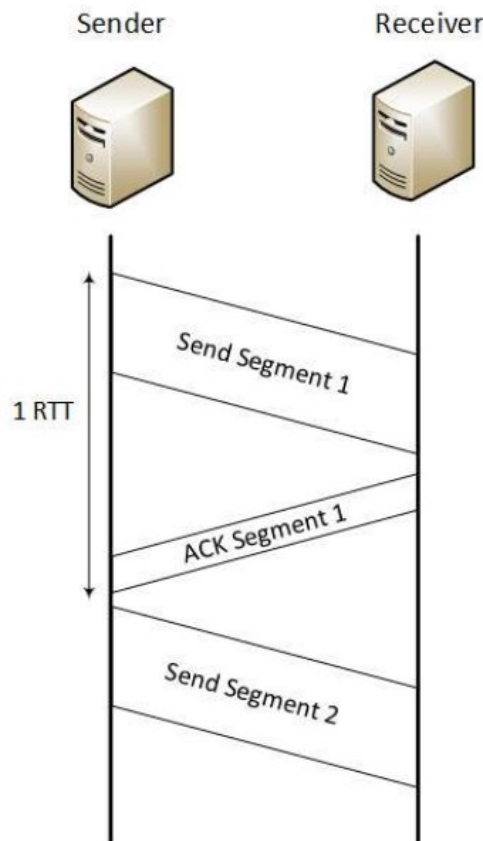
- Stop-and-Wait: Ein weiteres Segment darf erst nach Empfang der Bestätigung für das aktuelle Segment gesendet werden. Das Verfahren kann als Spezialfall eines Schiebefensterprotokolls mit Fenstergröße 1 betrachtet werden.
- Sliding Window: Ein Kreditmechanismus bestimmt, wie viele Segmente oder Bytes ohne Bestätigung gesendet werden dürfen. Die Fenstergröße kann in Segmenten, Bytes oder Vielfachen der Maximum Segment Size (MSS) angegeben werden.
- Schiebefensterverfahren werden im Fehlerfall in Go-Back-N und Selective Repeat unterschieden.

Verfahren	Daten ohne ACK	Reaktion auf Verlust	Wirkung
Stop-and-Wait	Genau ein Segment	Dasselbe Segment nach Timeout erneut senden	Einfach, aber bei großer RTT sehr geringer Durchsatz
Go-Back-N	Mehrere Segmente gemäß Fenster	Ab verlorenem Segment alle nachfolgenden Segmente erneut senden	Einfachere Empfängerverarbeitung, aber unnötige Wiederholungen
Selective Repeat	Mehrere Segmente gemäß Fenster	Nur fehlende Segmente erneut senden	Effizienter, benötigt Puffer und genauere Verlustinformation

Abgedeckte Folie: Arten von ARQ-Verfahren (PDF-Seite 10 / Folie 9).

3.3 Stop-and-Wait ARQ

- Erst nach Erhalt des ACKs kann das nächste Segment gesendet werden.
- Nach Ablauf eines Timeouts wird das Segment als verloren interpretiert und erneut gesendet.
- Der Durchsatz ist schlecht, weil zwischen den Segmenten hohe Wartezeiten entstehen können.
- Beispiel der Folie: Bei 100 ms RTT und 1460 Byte MSS sind höchstens ungefähr 10 Segmente pro Sekunde möglich. Das entspricht 14 600 Byte pro Sekunde.



Wartezeiten bei Stop&Wait

Abbildung 3: Wartezeiten bei Stop-and-Wait. Ein neues Segment wird erst nach dem ACK des vorherigen Segments gesendet (Original aus PDF-Seite 11 / Folie 10).

Berechnung des Beispiels: 100 ms entsprechen 0,1 s. Damit sind unter der vereinfachenden Annahme vernachlässigbarer Übertragungs- und Verarbeitungszeit etwa $1 / 0,1 = 10$ Übertragungszyklen pro Sekunde möglich. $10 \times 1460 \text{ Byte} = 14 600 \text{ Byte/s}$.

RTT: Round Trip Time ist die Zeit vom Absenden bis zum Eintreffen der zugehörigen Antwort beziehungsweise Bestätigung. Bei Stop-and-Wait begrenzt die RTT unmittelbar die Häufigkeit neuer Segmente.

Abgedeckte Folie: Stop-and-Wait ARQ (PDF-Seite 11 / Folie 10).

3.4 Prinzip der Schiebefensterverfahren

- Mehrere Segmente werden unmittelbar hintereinander gesendet, ohne für jedes einzelne zuerst auf ein ACK zu warten.
- Die Fenstergröße legt fest, wie viel insgesamt gesendet werden darf.

- Jedes gesendete Segment verbraucht Kredite entsprechend der Größe seiner Nutzdaten. Headerbytes werden für diesen Kredit nicht gezählt.
- Wenn kein Kredit mehr vorhanden ist, muss der Sender unterbrechen.
- Eintreffende ACKs geben Kredite frei, sodass der Sender fortfahren kann.

Kredit in Byte: Bei einem Fenster von 6000 Byte und 4000 Byte bereits unbestätigten Nutzdaten sind noch 2000 Byte neuer Nutzdaten sendbar. Eine Retransmission bereits angerechneter Daten benötigt nach dem Modell der Folien keinen neuen Kredit.

Abgedeckte Folie: Prinzip der Schiebefensterverfahren (PDF-Seite 12 / Folie 11).

3.5 Kumulative und selektive Bestätigungen

- Kumulative Bestätigung: Ein ACK bestätigt alles bis einschließlich eines bestimmten Segments beziehungsweise bei TCP alle Bytes vor der Acknowledge Number.
- Selektive Bestätigung: Eine Bestätigung bezeichnet genau ein Segment oder bestimmte Datenbereiche.
- Im fehlerfreien Ablauf macht die Unterscheidung für die Kreditfreigabe keinen wesentlichen Unterschied; jede neue Bestätigung gibt weiteren Kredit frei.
- Bei einer Lücke führen kumulative Bestätigungen zu Duplicate ACKs (DupACKs), weil wiederholt dieselbe nächste erwartete Position bestätigt wird.

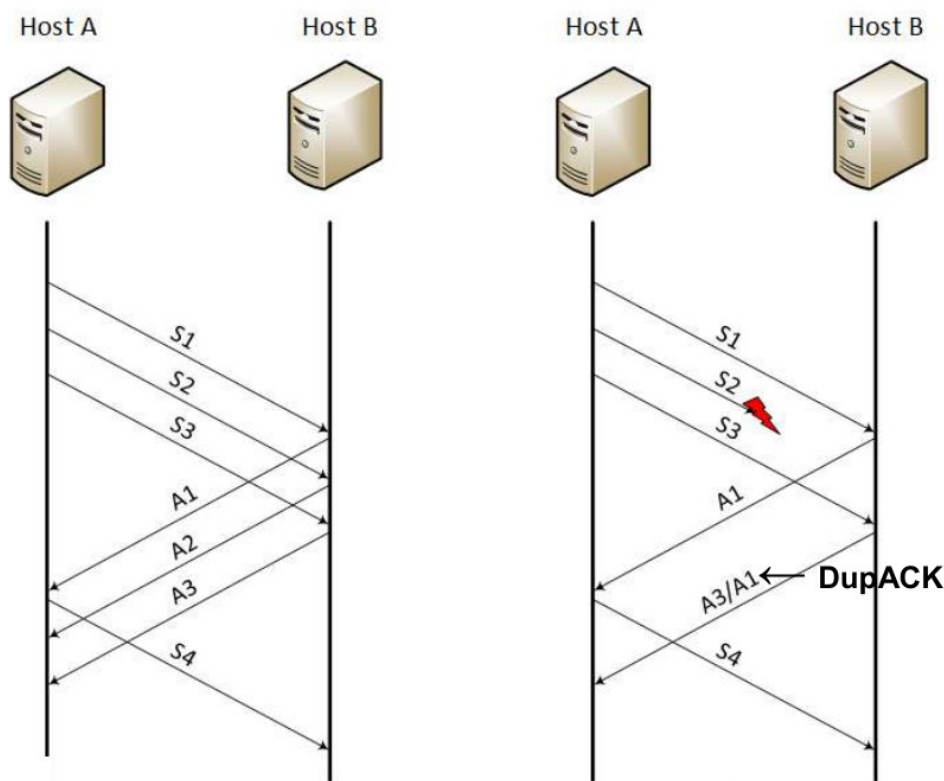


Abbildung 4: Sliding-Window-Ablauf ohne Fehler (links) und mit Verlust/Lücke (rechts). Die Lücke erzeugt wiederholte ACKs für denselben Stand (Original aus PDF-Seite 13 / Folie 12).

Beispiel für ein DupACK: Erwartet der Empfänger als Nächstes Byte 3001, erhält aber bereits spätere Bytes, bleibt seine kumulative Acknowledge Number 3001. Jedes weitere außer der Reihenfolge eintreffende Segment kann deshalb erneut ein ACK mit 3001 auslösen.

Abgedeckte Folie: Ablauf mit und ohne Fehler (PDF-Seite 13 / Folie 12).

3.6 Fehlerbehandlung im Schiebefenster

- Beim Ablauf eines Timeouts oder eventuell beim Empfang eines NAKs wird das entsprechende Segment als verloren betrachtet.
- Das verlorene Segment muss identifiziert werden. TCP verwendet dazu die Sequenznummer.
- Danach erfolgt eine Retransmission. Nach dem Kreditmodell der Folien benötigt sie keinen neuen Kredit, weil die Daten bereits zum ausstehenden Fenster gehören.
- Ist die Retransmission erfolgreich, kann der Empfänger die Daten verarbeiten und bei TCP in den Bytestrom einordnen.
- Der Sender erhält ein ACK. Dieses muss die erneut übertragenen Daten implizit bei kumulativer Bestätigung oder explizit bei selektiver Bestätigung identifizieren können.

In-Flight-Daten: Als In-Flight gelten gesendete, aber noch nicht bestätigte Nutzdaten. Eine Wiederholung erhöht nicht die Menge unterschiedlicher ausstehender Nutzdaten, auch wenn zusätzliche Bits über das Netz übertragen werden.

Abgedeckte Folie: Fehlerbehandlung in Schiebefensterverfahren (PDF-Seite 14 / Folie 13).

3.7 Go-Back-N und Selective Repeat

Go-Back-N:

- Nach einem Fehler wird ab dem verlorenen Segment alles erneut übertragen.
- Mit größerem Fenster wird das Verfahren zunehmend ineffizient, weil auch bereits korrekt übertragene nachfolgende Segmente wiederholt werden.

Selective Repeat:

- Nach einem Fehler wird nur das verlorene Segment erneut übertragen.
- Der Empfänger puffert bereits korrekt empfangene spätere Segmente. Nach erfolgreicher Retransmission kann er die Lücke schließen und die Reihenfolge wiederherstellen.

Einordnung von TCP: TCP ist kein reines Lehrbuch-Go-Back-N und kein reines Lehrbuch-Selective-Repeat. Es arbeitet byteorientiert mit einem Sliding Window und normalerweise kumulativen ACKs. Mit selektiven Bestätigungsinformationen können fehlende Bereiche genauer erkannt und gezielt wiederholt werden.

Abgedeckte Folie: Varianten von Schiebefensterverfahren (PDF-Seite 15 / Folie 14).

4. TCP Sliding Window

4.1 Offsetadressen und Sequenzraum des Senders

Die Folie teilt den Sequenzraum des Senders in vier Abschnitte ein:

- 1. Bereits gesendet und bestätigt.
- 2. Bereits gesendet, aber noch nicht bestätigt.
- 3. Noch nicht gesendet, darf aber innerhalb des Fensters gesendet werden.
- 4. Darf noch nicht gesendet werden, weil dieser Bereich außerhalb des aktuellen Fensters liegt.

Das Sendefenster besteht aus den Abschnitten 2 und 3.

Offsetadressen im Bytestrom

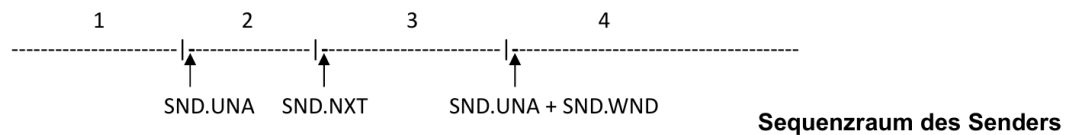


Abbildung 5: Sequenzraum des Senders mit $SND.UNA$, $SND.NXT$ und $SND.UNA + SND.WND$ (Original aus PDF-Seite 16 / Folie 15).

Variable	Bezeichnung	Bedeutung
$SND.UNA$	Send Unacknowledged	Sequenznummer des ersten Bytes, das noch nicht bestätigt wurde. Alles davor ist bestätigt.
$SND.NXT$	Send Next	Sequenznummer des nächsten neu zu sendenden Bytes.
$SND.WND$	Send Window	Aktuell zugelassene Fenstergröße für den Sender.
$SND.UNA + SND.WND$	Rechte Fenstergrenze	Erste Position außerhalb des derzeit zulässigen Sendebereichs.

Freier Kredit: Die bereits ausstehende Datenmenge ist $SND.NXT - SND.UNA$. Der noch verfügbare Kredit beträgt vereinfacht $SND.WND - (SND.NXT - SND.UNA)$, sofern das Ergebnis positiv ist.

Abgedeckte Folie: Offsetadressen im Bytestrom (PDF-Seite 16 / Folie 15).

4.2 Ablauf der Datenübertragung

- Die relative Sequenznummer bestimmt den Offset eines Bytes im Nutzdatenstrom.
- Sendet ein Host jetzt ein neues Segment, besitzt dieses Segment die Sequenznummer $SND.NXT$.
- Möchte der Empfänger alle bis dahin offenen Daten bestätigen, verwendet er als Acknowledge Number ebenfalls den Wert, der beim Sender dem nächsten Byte entspricht, also $SND.NXT$ nach Einbeziehung der gesendeten Nutzdaten.

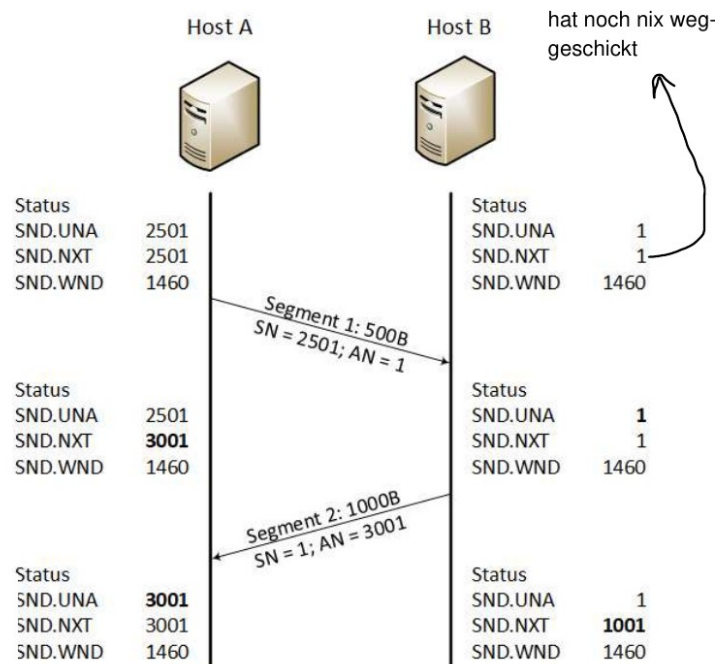


Abbildung 6: Beispielhafter Verlauf der Zustandsvariablen. Host A sendet 500 Byte mit SN 2501; Host B antwortet mit 1000 Byte und bestätigt mit AN 3001 (Original aus PDF-Seite 17 / Folie 16).

Aus dem Diagramm lassen sich folgende Zustandsänderungen ablesen:

- Host A startet mit SND.UNA = 2501, SND.NXT = 2501 und SND.WND = 1460.
- Host B startet in seiner eigenen Senderichtung mit SND.UNA = 1, SND.NXT = 1 und SND.WND = 1460; die handschriftliche Notiz weist darauf hin, dass Host B noch nichts gesendet hat.
- Host A sendet Segment 1 mit 500 Byte, SN = 2501 und AN = 1. Danach steigt A.s SND.NXT auf 3001, während SND.UNA zunächst 2501 bleibt.
- Host B sendet Segment 2 mit 1000 Byte, SN = 1 und AN = 3001. Mit AN = 3001 bestätigt B die 500 Byte von A.
- Nach der Bestätigung steigt bei Host A SND.UNA auf 3001. Bei Host B steigt SND.NXT nach 1000 gesendeten Bytes auf 1001.

Zusatznotiz der Folie zur MSS: Die Folie nennt 1460 Byte als Maximum Segment Size und verweist auf IP-Header. 1460 Byte ist der typische TCP-MSS-Wert bei einer Ethernet-MTU von 1500 Byte sowie 20 Byte IPv4-Header und 20 Byte TCP-Header ohne Optionen. Der Wert ist nicht in jedem Netz und nicht bei jeder Headerkombination identisch.

Abgedeckte Folie einschließlich kleiner Zusatznotiz: Ablauf der Datenübertragung (PDF-Seite 17 / Folie 16).

4.3 Festlegung der Fenstergröße und Channel Utilization

- Go-Back-N und Selective Repeat verwenden im einfachen Modell feste Fenstergrößen. Daraus ergibt sich die Frage nach der optimalen Fenstergröße.
- Im fehlerfreien Fall soll die Channel Utilization, also genutzte Übertragungszeit geteilt durch Gesamtzeit, den Wert 1 erreichen. Dann gibt es keine Leerlaufzeiten.
- Bei End-to-End-Verbindungen kann die RTT lang sein. Um die Leitung vollständig auszulasten, kann ein großes Fenster erforderlich sein.
- Sehr große Fenster sind bei Fehlern problematisch und können Netz oder Empfänger überfordern.

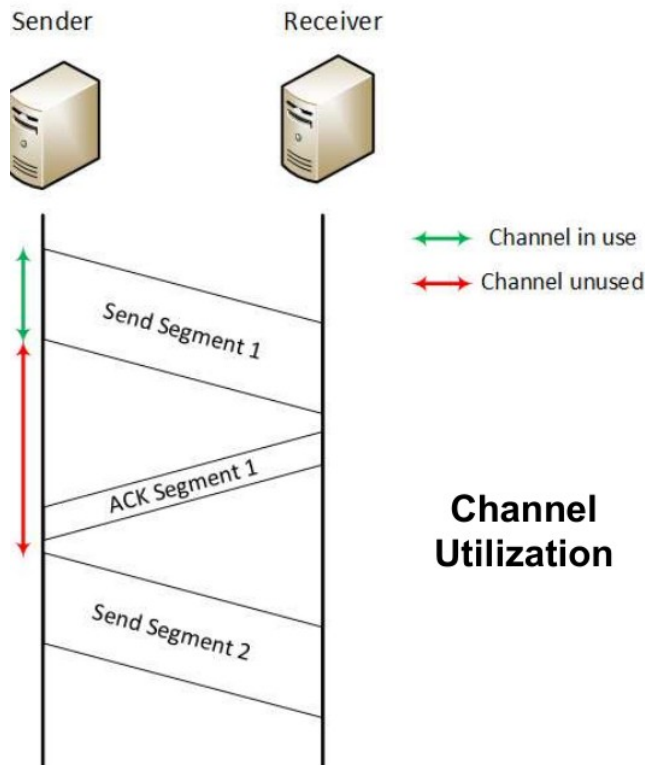


Abbildung 7: Channel Utilization. Die grüne Markierung zeigt genutzte, die rote Markierung ungenutzte Kanalzeit innerhalb eines Zyklus (Original aus PDF-Seite 18 / Folie 17).

Die kleinen Zusatznotizen der Folie lauten inhaltlich:

- Cycle Time = RTT + Reaktionszeit der Computer.
- Die Fenstergröße kann mindestens so gewählt werden, dass über eine RTT hinweg kontinuierlich Daten gesendet werden können. Das ermöglicht eine hohe Auslastung.
- Große Fenster können dennoch problematisch sein, beispielsweise wenn das Netz oder der Empfänger überfordert wird.

Bandbreiten-Verzögerungs-Produkt: Für eine kontinuierliche Übertragung muss das Fenster mindestens ungefähr so viele Bytes umfassen, wie während einer RTT über den Pfad übertragen werden können. Dieser Wert ergibt sich aus Datenrate x RTT. Die Folien beschreiben denselben Zusammenhang über die Forderung nach einer Channel Utilization von 1.

Abgedeckte Folie einschließlich Zusatznotizen: Festlegung der Fenstergröße (PDF-Seite 18 / Folie 17).

4.4 Variable Fenstergrößen bei TCP

- TCP verwendet variable Fenstergrößen. Sie reagieren sowohl auf Fehlersituationen als auch auf Lastsituationen.
- Flow Control: Der Empfänger gibt über das TCP-Headerfeld WND explizit an, wie viel Empfangspuffer verfügbar ist. Er kann diesen Wert herunterregeln.
- Congestion Control: Fehlende oder auffällige Bestätigungen dienen dem Sender als implizites Signal für mögliche Überlast oder Verlust. Die Fehlerursache ist für die grundlegende Reaktion nicht sicher unterscheidbar.
- Das Minimum aus Flow-Control-Grenze und Congestion-Control-Grenze wird als tatsächliches Sendefenster beziehungsweise Kreditrahmen verwendet.

Mechanismus	Informationsquelle	Schützt primär	Fensterbezeichnung
Flow Control	Explizites WND-Feld des Empfängers	Empfangspuffer und Empfänger	Receiver/Advertised Window, häufig rwnd
Congestion Control	Implizite Rückmeldung aus ACKs, Verlust und Timeouts	Netzpfad vor Überlast	Congestion Window, cwnd

Zentrale Beziehung: Effektives Sendefenster = $\min(\text{rwnd}, \text{cwnd})$. Der Sender darf außerdem nur den noch freien Teil dieses Fensters mit neuen Daten füllen.

Abgedeckte Folie: Fenstergrößen bei TCP (PDF-Seite 19 / Folie 18).

4.5 TCP Congestion Control und Slow Start

- Congestion Control ist im Foliensatz der primäre Begrenzungsmechanismus, der verhindert, dass Fenster unkontrolliert groß werden.
- Das Congestion Window ist immer auf einen endlichen Wert gesetzt.
- Das Congestion Window ist adaptiv und wird bei erkannten Fehlern verkleinert, unabhängig von der tatsächlichen Fehlerursache.
- Der Sender beginnt mit einer geringen Fenstergröße und vergrößert sie schrittweise, solange keine Fehler auftreten.
- Im vereinfachten Folienmodell startet Slow Start mit 1 MSS, typischerweise 1460 Byte.
- Die Vergrößerung wird bei jedem bestätigten Segment durchgeführt.
- Die Zusatznotiz formuliert: 1 MSS entspricht zunächst faktisch Stop-and-Wait; bei jedem Acknowledge wächst die Window Size.

Präzisierung des exponentiellen Wachstums: Im vereinfachten Slow-Start-Modell erhöht jedes ACK das Congestion Window ungefähr um 1 MSS. Wenn innerhalb einer RTT alle Segmente bestätigt werden, trifft eine entsprechend größere Zahl von ACKs ein. Dadurch verdoppelt sich cwnd ungefähr pro RTT, nicht durch ein einzelnes ACK.

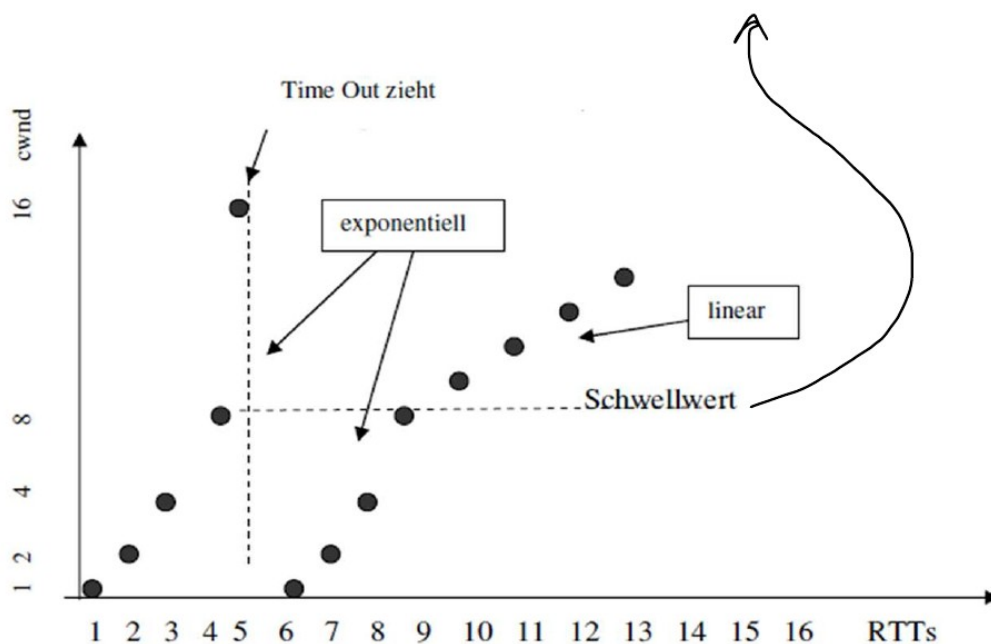
Vereinfachte Rechenregel: Slow Start: $\text{cwnd} = \text{cwnd} + \text{MSS}$ pro ACK. Oberhalb des Schwellenwerts wird das Wachstum linear; als typische vereinfachte Regel ergibt sich insgesamt ungefähr +1 MSS pro RTT.

Abgedeckte Folie einschließlich Zusatznotizen: TCP Congestion Control (PDF-Seite 20 / Folie 19).

4.6 Verlauf des Congestion Windows

- Nach einem Fehler beginnt TCP im vereinfachten Modell der Folie wieder mit 1 MSS.
- Das Wachstum ist zunächst exponentiell.
- Ab einem Schwellenwert wächst das Fenster linear.
- Die handschriftliche Diagrammnotiz setzt den Schwellenwert auf die Hälfte der Fenstergröße an der Stelle, an der der Fehler auftrat.
- Im Diagramm löst ein Timeout den Einbruch des Congestion Windows aus.

Schwellenwert: hälft vom window size von da wo der fehler passiert ist



Zeitlicher Verlauf des Congestion Windows

Abbildung 8: Zeitlicher Verlauf des Congestion Windows. Nach einem Timeout fällt cwnd auf 1; bis zum Schwellenwert wächst es exponentiell, danach linear (Original aus PDF-Seite 21 / Folie 20).

Die kleinen Zusatznotizen unter der Folie enthalten folgende Aussagen:

- Es gibt ein Verfahren, das adaptiv auf Fehler reagiert.
- Verwendet wird der kleinere Wert aus Flow-Control-Limit des Empfängers und Congestion-Control-Limit.
- Wenn das vom Empfänger erlaubte Fenster größer wird, folgt die tatsächlich nutzbare Fenstergröße weiterhin der Logik der Congestion Control, solange cwnd der kleinere Wert ist.

Korrekturhinweis zum 16-Bit-WND-Feld: Die Folie nennt als theoretisches Maximum „65535 MSS“. Das 16-Bit-WND-Feld codiert ohne Skalierung jedoch maximal 65 535 Byte, nicht 65 535 MSS. In Darstellungen kann cwnd in MSS-Einheiten angegeben werden; dies ist von der Codierung des WND-Feldes zu unterscheiden. TCP-Optionen können außerdem eine Skalierung des angekündigten Fensters ermöglichen.

Vereinfachte Reaktion auf Timeout: 1. Schwellenwert auf etwa die Hälfte der vorherigen ausstehenden Fenstergröße setzen. 2. cwnd auf 1 MSS zurücksetzen. 3. Erneut exponentiell bis zum Schwellenwert wachsen. 4. Danach linear weiterwachsen. Diese Darstellung entspricht dem in der Folie gezeigten Lehrmodell.

Abgedeckte Folie einschließlich Diagramm- und Zusatznotizen: Verlauf des Congestion Windows (PDF-Seite 21 / Folie 20).

5. Kompakte Zusammenfassung

Thema	Kernaussage
TCP-Dienst	Zuverlässiger, verbindungsorientierter End-to-End-Transport eines Bytestroms über IP.

Thema	Kernaussage
Segmentierung	Der Sender zerlegt den Bytestrom in Segmente; der Empfänger reassembliert ihn anhand von Sequenznummern.
Zuverlässigkeit	ACKs, Timeouts, Retransmissions, Sequenznummern, Reihenfolgeprüfung, Duplikaterkennung und Prüfsumme.
Sicherheitsgrenze	TCP selbst bietet keine Verschlüsselung und keine Authentifizierung.
Handshake	$\text{SYN}(x)$, $\text{SYN+ACK}(y, x+1)$, $\text{ACK}(x+1, y+1)$.
ARQ	Korrekte Daten werden bestätigt; fehlende oder fehlerhafte Daten werden erneut übertragen.
Stop-and-Wait	Nur ein Segment ohne ACK; geringer Durchsatz bei großer RTT.
Sliding Window	Mehrere Bytes/Segmente dürfen gemäß Fenster als Kredit gleichzeitig ausstehen.
Kumulative ACKs	AN bezeichnet das erste noch fehlende Byte; Lücken führen zu DupACKs.
Go-Back-N	Ab dem verlorenen Segment werden alle nachfolgenden Daten wiederholt.
Selective Repeat	Nur fehlende Daten werden wiederholt; spätere korrekte Daten werden gepuffert.
Senderzustand	SND.UNA = erstes unbestätigtes Byte; SND.NXT = nächstes neu zu sendendes Byte; SND.WND = Fenster.
Flow Control	Empfänger begrenzt über WND die ausstehenden Daten.
Congestion Control	Sender passt cwnd anhand von Verlust- und ACK-Signalen an.
Effektives Fenster	$\min(\text{rwnd}, \text{cwnd})$.
Slow Start	Im Folienmodell Start mit 1 MSS; ungefähr exponentielles Wachstum pro RTT.
Nach Schwelle	Lineares Wachstum; nach Timeout Rückfall und neuer Anlauf.

Prüfungsrelevante Unterscheidung: Flow Control schützt den Empfänger; Congestion Control schützt den Netzpfad. Beide begrenzen gleichzeitig den Sender. Zuverlässigkeit entsteht durch ARQ, während die Fenstersteuerung zusätzlich Durchsatz und Überlast beeinflusst.

6. Begriffs- und Formelübersicht

Begriff	Definition
ACK	Acknowledge; positive Bestätigung korrekt empfangener Daten.
AN	Acknowledge Number; Nummer des ersten noch nicht bestätigten beziehungsweise als Nächstes erwarteten Bytes.
ARQ	Automated Repeat Request; Bestätigungs- und Wiederholungsmechanismus.

Begriff	Definition
cwnd	Congestion Window; senderseitige Grenze aus der Überlaststeuerung.
DupACK	Duplicate ACK; wiederholte Bestätigung derselben Acknowledge Number, typischerweise bei einer Lücke.
Flow Control	Anpassung der Sendemenge an den verfügbaren Empfangspuffer.
Congestion Control	Anpassung der Sendemenge an die vermutete Belastbarkeit des Netzpfs.
MSS	Maximum Segment Size; maximale TCP-Nutzdatenmenge pro Segment für einen Pfad beziehungsweise eine Verbindung.
NAK	Negative Acknowledgement; ausdrückliche Meldung eines Fehlers.
Retransmission	Erneute Übertragung nicht bestätigter oder als verloren erkannter Daten.
RTT	Round Trip Time; Zeit für Hinweg und Rückmeldung.
rwnd/WND	Vom Empfänger angekündigtes Fenster zur Flow Control.
SN	Sequence Number; Nummer des ersten Nutzdatenbytes eines TCP-Segments.
SND.UNA	Erstes noch nicht bestätigtes Byte im Senderzustand.
SND.NXT	Nächstes neu zu sendendes Byte.
SND.WND	Aktuelle Sendefenstergröße.
ssthresh	Schwellenwert zwischen exponentiellem Slow Start und linearem Wachstum.
Timeout	Ablauf einer Zeitüberwachung ohne ausreichende Bestätigung.

6.1 Zentrale Beziehungen

- Effektives Sendefenster = $\min(rwnd, cwnd)$.
- Ausstehende neue Daten = $SND.NXT - SND.UNA$.
- Freier Kredit = effektives Sendefenster - ausstehende neue Daten.
- Stop-and-Wait-Durchsatz im vereinfachten Modell = MSS / RTT .
- Für volle Auslastung benötigtes Fenster ungefähr = $Datenrate \times RTT$.
- Slow Start im Folienmodell: ungefähr Verdopplung von cwnd pro RTT.
- Nach dem Schwellenwert: ungefähr lineares Wachstum um 1 MSS pro RTT.

Abschluss: Die zentrale Idee des gesamten Kapitels ist die kontrollierte Menge unbestätigter Daten. Sequenznummern und ACKs sichern die korrekte Reihenfolge und Erkennung von Verlusten. Das Sliding Window verbessert den Durchsatz. Flow Control und Congestion Control begrenzen das Fenster so, dass weder Empfänger noch Netzpfad überlastet werden.

Quellen- und Abbildungshinweis

Grundlage dieser Ausarbeitung ist der bereitgestellte Foliensatz:

Armin Veichtlbauer: VO Computernetzwerke - Kapitel 6, SS 2026, 21 PDF-Seiten.

Die eingefügten Abbildungen sind direkte Ausschnitte aus den jeweiligen Folien. Die ergänzenden Erklärungen wurden in einfacher sachlicher Form formuliert. Zwei technisch missverständliche Stellen des Foliensatzes wurden im Skript ausdrücklich als Korrekturhinweise gekennzeichnet: die Absenderzuordnung der dritten Handshake-Nachricht und die Einheit des 16-Bit-WND-Feldes.